Übungsblatt 6: Das Steuerwerk

Mikrocomputertechnik 1

Musterlösung

Zielsetzung und Vorgehen

Sie leiten Steuersignale aus der Befehlswirkung her, unterscheiden Haupt- und ALU-Steuerung und bestimmen konkrete Folgen falscher Freigaben. Teil A ist ohne Simulator lösbar und übt Klausuraufgaben. Teil B vergleicht Vorhersagen mit Ripes.

Orientierung: Teil A etwa 60 Minuten und 50 Punkte zur Selbsteinschätzung, Teil B etwa 25 Minuten. Begründen Sie die Entscheidungen; eine Liste von Bitwerten allein genügt bei Erklärungsaufgaben nicht.

Vereinbartes Modell

Es gilt das Single-Cycle-Lehrmodell aus Vorlesung 5 und 6 mit der in Einheit 6 ergänzten `addi`-Unterstützung:

- Unterstützt: `add`, `sub`, `and`, `or`, `lw`, `sw`, `beq`, `addi`.
- Gültige Codierungen, 32-Bit-Daten, unkomprimierte Vier-Byte-Instruktionen, byteadressierter Speicher, Little Endian.
- Lesen kombinatorisch, Übernahme an der steigenden Flanke mit Freigabe. Alle genannten Speicheradressen sind gültig, beschreibbar und wortausgerichtet.
- `ALUSrc`: 0 wählt RD2, 1 den vorzeichenerweiterten Immediate.
- `MemtoReg`: 0 wählt das ALU-Ergebnis, 1 die Speicherlesedaten. X bedeutet eine nachweislich unerhebliche Auswahl.
- `ALUOp`: `00` erzwingt ADD, `01` SUB für `beq`, `10` wertet die gültigen Register-Funktionsfelder aus.
- `ALUControl`: `0000` AND, `0001` OR, `0010` ADD, `0110` SUB. Die Codes sind Kurskonventionen.
- PCSrc $=$ Branch $\wedge$ Zero. ImmGen liefert den fertigen Branch-Byteoffset ohne weiteren Shift.

Registernummern: `t0` $=$ `x5`, `t1` $=$ `x6`, `t2` $=$ `x7`, `s1` $=$ `x9`. Alle unabhängigen Fälle beginnen mit den jeweils angegebenen Anfangswerten.

Teil A: Papieraufgaben

Aufgabe 1: Hauptsteuerung herleiten (12 Punkte)

1. Füllen Sie die Quellen- und Freigabetabelle aus. Verwenden Sie X nur mit begründeter Unerheblichkeit. (6 P.)

Befehl	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch
`add`						
`lw`						
`sw`						
`beq`						
`addi`						

2. Geben Sie für jeden der fünf Befehle das zweibitige ALUOp an. Warum braucht `addi` dieselbe Kategorie wie `lw` und nicht dieselbe wie `add`? (3 P.)
3. Begründen Sie das X bei `sw` über Schreibport und Datenweg. Dürfte stattdessen `MemWrite` mit X bezeichnet werden? (2 P.)
4. Wie viele benannte Hauptsignale und wie viele Bits enthält die gesamte Hauptsteuerzeile? Gehört PCSrc als unabhängiger Ausgang dazu? (1 P.)

Musterlösung Aufgabe 1

Befehl	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch
`add`	0	0	1	0	0	0
`lw`	1	1	1	1	0	0
`sw`	1	X	0	0	1	0
`beq`	0	X	0	0	0	1
`addi`	1	0	1	0	0	0

2. In Tabellenreihenfolge: `10`, `00`, `00`, `01`, `00`. Bei `addi` steht Addition schon durch die gültige Befehlserkennung fest. Die oberen Bits sind Immediate-Bits, keine Register-Funktionskennung für Plus oder Minus.
3. `RegWrite = 0` sperrt den Registerschreibport. `MemtoReg` wählt nur dessen Datenquelle; die Speicher-Schreibdaten kommen getrennt aus RD2. `MemWrite` darf kein X sein, weil sein Wert darüber entscheidet, ob der Store überhaupt schreibt.
4. Sieben benannte Hauptsignale mit acht Bits, weil ALUOp zwei Bit hat. PCSrc wird aus Branch und Zero abgeleitet.

Aufgabe 2: ALUOp, ALUControl und Bit 30 (10 Punkte)

Gegeben sind unabhängige Fälle mit `t1 = 7` und `t2 = 3`:

Fall	Befehl	Relevante Felder
A	`sub t0, t1, t2`	Opcode `0110011`, funct3 `000`, funct7 `0100000`
B	`or t0, t1, t2`	Opcode `0110011`, funct3 `110`, funct7 `0000000`
C	`addi t0, t1, -5`	Opcode `0010011`, funct3 `000`, Immediate `1111 1111 1011`

1. Bestimmen Sie ALUOp, ALUControl, die Quelle des zweiten ALU-Operanden und den neuen Inhalt von `t0` für alle drei Fälle. (6 P.)
2. Bit 30 ist in A und C gesetzt. Warum verlangt es nur in A eine Subtraktion? (2 P.)
3. Ein anderes Muster hat den Register-Opcode, funct3 `000` und funct7 `0000001`. Darf unser Decoder allein wegen Bit 30 gleich 0 auf `add` schließen? Begründen Sie die Antwort. (2 P.)

Musterlösung Aufgabe 2

Fall	ALUOp	ALUControl	Quelle B	Ergebnis
A	`10`	`0110`	RD2 = 3	`t0 = 4`
B	`10`	`0001`	RD2 = 3	`t0 = 7`
C	`00`	`0010`	Immediate -5	`t0 = 2`

A: $7 - 3 = 4$. B: Die unteren vier Bits `0111` ODER `0011` ergeben `0111`, alle oberen Bits bleiben null. C: $7 + (-5) = 2$; der erweiterte Immediate ist `0xFFFFFFFB`.

2. A ist eine gültige Registeroperation mit Kategorie `10`; Bit 30 gehört zum funct7-Feld und unterscheidet hier `sub` von `add`. C ist `addi` mit Kategorie `00`; Bit 30 gehört zur Konstanten und wird nicht als Subtraktionskennung ausgewertet.
3. Nein. Das gesamte gültige funct7 für `add` ist `0000000`. Das abweichende Muster ist kein unterstützter Fall unseres Subsets. Der reduzierte Bit-30-Test setzt bereits legale Befehle voraus.

Aufgabe 3: Boolesche Steuerbedingungen (10 Punkte)

R, L, S, B und I sind gegenseitig ausschließende Ein-Bit-Signale für gültig erkannte Registeroperationen, `lw`, `sw`, `beq` und `addi`.

1. Leiten Sie eine Boolesche Formel für RegWrite und eine für ALUSrc her. Verwenden Sie $\wedge$, $\vee$ und bei Bedarf den Überstrich zur Negation. (4 P.)
2. Zeichnen Sie RegWrite mit einem rechteckigen ODER-Gatter nach der verwendeten IEC-Darstellung. Beschriften Sie Ein- und Ausgang und die Bitbreiten. (2 P.)
3. Ergänzen Sie die folgenden Funktionen für jede gültige Befehlsgruppe. Warum sind beide zulässig? (3 P.)

$$\text{MemtoReg}_a = L, \qquad \text{MemtoReg}_b = L \vee S$$

Gruppe	R	L	S	B	I
MemtoReg a					
MemtoReg b					

4. Bedeutet X in der Steuertabelle einen hochohmigen Ausgang? Erklären Sie den Unterschied. (1 P.)

Musterlösung Aufgabe 3

1. RegWrite $= R \vee L \vee I$, weil diese Gruppen Register schreiben. ALUSrc $= L \vee S \vee I$, weil diese Gruppen das Immediate als zweiten ALU-Operanden benötigen.
2. Drei Ein-Bit-Eingänge R, L und I führen in ein Rechteck mit dem Symbol >=1. Ein Ein-Bit-Ausgang trägt RegWrite. Die Leitungen sind rechtwinklig zu führen. Es ist kein Datenwort und kein Taktsignal.
3. In Gruppenreihenfolge:

Gruppe	R	L	S	B	I
MemtoReg a	0	1	0	0	0
MemtoReg b	0	1	1	0	0

Nur beim Store unterscheiden sich die Werte. Dort ist RegWrite null, daher wird keine Rückschreibquelle übernommen. Der Speicher-Schreibwert kommt aus RD2 und bleibt gleich.
4. Nein. X bezeichnet Entwurfsfreiheit bei unerheblicher Auswahl. Ein hochohmiger Ausgang treibt elektrisch keinen eindeutigen Pegel. Das sind verschiedene Begriffe.

Aufgabe 4: Fehlerfolgen exakt bestimmen (10 Punkte)

Zwei voneinander unabhängige, gedachte Decoderfehler im Lehrmodell:

Fall A: Speicherfreigabe bei Addition

```
add t0, t1, t2        # t1 = 0x2000; t2 = 4
```

Nur `MemWrite` ist falsch auf 1 gesetzt. Wort an `0x2004` vorher: 99. Alle übrigen Signale entsprechen einem korrekten `add`.

1. Welche Adresse und welche Schreibdaten liegen am Datenspeicher? (2 P.)
2. Welche Werte enthalten `t0` und das Wort an `0x2004` nach der Flanke? Geben Sie außerdem die vier Speicherbytes in aufsteigender Adressreihenfolge an. (2 P.)

Fall B: Registerfreigabe bei Store

```
sw t1, 12(s1)        # s1 = 0x2000; t1 = 37; x12 = 9
```

Hier sind `RegWrite = 1` und `MemtoReg = 0` gesetzt. Alle anderen Signale entsprechen einem korrekten Store. Die Bits 11:7 der Instruktion enthalten die unteren fünf Offsetbits, also `01100`; der Registeradressport ist fest mit diesen Positionen verbunden.

3. Bestimmen Sie den Speicher-Schreibort und den geschriebenen Wert. Welches Register wird zusätzlich beschrieben und mit welchem Wert? (3 P.)
4. Warum sind die Folgen in beiden Fällen nicht „zufällige Zellen mit Müll“? Warum setzt das Don’t Care bei MemtoReg eine korrekte Registerfreigabe voraus? (3 P.)

Musterlösung Aufgabe 4

1. A: ALU-Adresse $\texttt{0x2000} + 4 = \texttt{0x2004}$, Schreibdaten aus RD2 gleich 4.
2. `t0 = 0x2004`; Speicherwort an `0x2004` gleich 4 statt 99. Bytes: `04 00 00 00` (Hex, Little Endian).
3. B: Speicheradresse $\texttt{0x2000} + 12 = \texttt{0x200C}$, Speicher-Schreibwert 37. `01100` adressiert `x12`. Wegen `RegWrite = 1` und `MemtoReg = 0` erhält `x12` zusätzlich das ALU-Ergebnis `0x200C` statt 9.
4. Adresse und Schreibdaten sind durch den Datenpfad und die Anfangswerte bestimmt. Ein Absturz hängt von der späteren Verwendung ab und ist nicht garantiert. MemtoReg ist beim korrekten Store nur deshalb unerheblich, weil RegWrite null ist. Nach Verletzung dieser Voraussetzung kann die Rückschreibquelle beobachtbar werden.

Aufgabe 5: Branch-Freigabe und Entscheidung (8 Punkte)

Der aktuelle Befehl liegt bei `PC = 0x100`. Sein vollständiger Branch-Byteoffset zum Label `ziel` beträgt $+12$.

1. Ergänzen Sie die Tabelle. C ist eine unabhängige Registeroperation an derselben PC-Adresse. (4 P.)

Fall	Befehl und Operanden	Branch	Zero	PCSrc	Neuer PC
A	`beq t0, t1, ziel`; 5 und 5				
B	`beq t0, t1, ziel`; 5 und 6				
C	`sub t0, t1, t2`; 5 und 5				

2. Welche ALUOp- und ALUControl-Werte gelten in A und B? Welche in C? (2 P.)
3. Warum bleibt Branch auch beim nicht genommenen Sprung 1? Warum wird der Byteoffset hier nicht nochmals verschoben? (2 P.)

Musterlösung Aufgabe 5

Fall	Branch	Zero	PCSrc	Neuer PC
A	1	1	1	`0x10C`
B	1	0	0	`0x104`
C	0	1	0	`0x104`

2. A und B: ALUOp `01`, ALUControl `0110`, Subtraktion zum Vergleich. C: ALUOp `10`, ALUControl ebenfalls `0110`, aus den gültigen `sub`-Funktionsfeldern.
3. Branch kennzeichnet die Instruktionsart, nicht das Vergleichsergebnis. PCSrc verbindet Branch mit Zero. ImmGen hat das implizite Bit 0 schon eingefügt und liefert den fertigen Byteoffset. Das Ziel ist der alte PC plus zwölf, kein zweiter Shift und keine Addition zu PC plus vier.

Teil B: Beobachtung in Ripes

Aufgabe 6: Ein Programm, mehrere Steueraufgaben

Verwenden Sie `beispiele/vl6_mux_beobachter.s`. Wählen Sie einen RV32-Single-Cycle-Prozessor, deaktivieren Sie C und notieren Sie Version und Prozessorbezeichnung. Lesen Sie die tatsächliche Adresse von `slot` als B ab.

Die Initialisierung `la s1, slot` kann mehrere Maschineninstruktionen erzeugen. Beginnen Sie bei `Beobachtung`. Halten Sie vor den Exit-Anweisungen bei `Pruefpunkt` an; die Simulator-Dienstkonvention ist für die Aufgaben nicht erforderlich.

Vor jedem Clock-Schritt: PC und Befehl, Operanden, ausgewählte Registerschreibquelle und Register-/Speicherfreigaben protokollieren. Nach dem Schritt: gespeicherte Register, Speicherwort und PC prüfen. Die dann sichtbaren Leitungen können schon zur nächsten Instruktion gehören.

Vorhersage und Protokoll

1. Ergänzen Sie zunächst die erwarteten Zustandsänderungen. (ohne Punkte)

Befehl	Nach Clock erwartet	Nach Clock beobachtet
`addi t0, x0, 5`		
`add t1, t0, t0`		
`sw t1, 0(s1)`		
`lw t2, 0(s1)`		
erster `beq`		
`addi t3, x0, 1`		
zweiter `beq`		

2. Welche konkrete Register-Rückschreibquelle zeigt Ripes beim Store und beim Branch? Warum kann ein angezeigter Auswahlwert mit X in der Papiertabelle vereinbar sein?
3. Welches Bauteil berechnet den Branch-Vergleich, welches das Ziel? Vergleichen Sie die Funktionen mit dem Lehrmodell, ohne gleiche interne Codes vorauszusetzen.
4. Dokumentieren Sie die Register- und Speicherwerte am Prüfpunkt und begründen Sie, warum der Markerwert 99 nicht ausgeführt wird.

Musterlösung Aufgabe 6: funktionale Erwartungen

Befehl	Nach Clock erwartet
`addi t0, x0, 5`	`t0 = 5`
`add t1, t0, t0`	`t1 = 10`
`sw t1, 0(s1)`	Wort an B ist 10
`lw t2, 0(s1)`	`t2 = 10`
erster `beq`	nicht genommen, PC zur nächsten Instruktion
`addi t3, x0, 1`	`t3 = 1`
zweiter `beq`	genommen, PC zum Label `Pruefpunkt`

2. Ein konkreter Quellenwert ist bei gesperrter Registerübernahme unerheblich. Ein Mehrwege-MUX kann andere Codes als unser MemtoReg verwenden. X ist keine notwendige Simulatoranzeige.
3. Die Bauteile und Ports sind anhand der verwendeten Version zu benennen. Ein eigener Registervergleicher und eine Haupt-ALU für das Branch-Ziel erfüllen dieselbe Funktion wie der getrennte Subtraktions-/Addiererweg unserer Zeichnung.
4. Am Prüfpunkt sind `t0 = 5`, `t1 = 10`, `t2 = 10`, `t3 = 1` und das Wort an B gleich 10. Der erste Branch vergleicht 5 mit 10 und fällt durch; der zweite vergleicht 10 mit 10 und überspringt die Instruktion, die 99 nach `t3` schreiben würde. Konkrete Portwerte und Adressen sind im eigenen Protokoll zu ergänzen.